Informe Proyecto 2: Juego de reflejos en Assembly sobre Pochoco SoC

Arquitectura de Computadores · Semestre 2026-2 Universidad de los Andes — Facultad de Ingeniería y Ciencias Aplicadas · Profesor: Jorge Gómez Mir

Integrantes: Lucas Fernández · Maximiliano Ferrando · Renato Rivadeneira · Matías Ureta

Repositorio: github.com/Max-Ferrando/Proyecto2_juego_Grupo7

1. OBJETIVO Y ALCANCE

Se implementó un juego de reflejos que corre como programa en assembly RISC-V sobre el Pochoco SoC, instanciado en una FPGA Lattice iCE40 HX1K de la Nandland Go Board. En cada ronda el sistema enciende un LED objetivo elegido de forma pseudoaleatoria y mide cuánto tarda el jugador en presionar el botón correspondiente. El tiempo se muestra en décimas de segundo; tras diez respuestas correctas se muestra el promedio.

El proyecto tiene tres partes que se desarrollaron juntas:

- **Hardware:** un top-level propio (`game_top.v`) y dos modificaciones al periférico del SoC: un contador de ciclos accesible por software y un sincronizador para los botones.
- **Software:** el juego completo en assembly (`sw/game.s`), 153 palabras de código de máquina.
- **Assembler:** un assembler propio en Python (`tools/rvasm.py`) que traduce `game.s` a `game.hex` sin invocar ningún assembler RISC-V externo.

La restricción central es el procesador: el Espino Core implementa RV32E (solo 16 registros), no tiene multiplicación ni división, y en esta versión los desplazamientos (shifts) están deshabilitados y se ejecutan como sumas. Todas las decisiones del programa se explican a partir de esa restricción.

1.1. Cumplimiento de los requisitos del enunciado

La siguiente tabla resume cómo se cumple cada requisito y en qué sección se documenta.

Requisito del enunciado	Cómo se cumple	Sección
4 LEDs encendidos ≥ 3 s al iniciar cada ronda	delay de 75.000.000 ciclos medido con el contador, más 20 ms de antirrebote (3,02 s)	4.2
LED objetivo pseudoaleatorio, no repetible	Generador congruencial sembrado con el ciclo exacto de una pulsación humana	5.5
Medición desde el instante lógico de inicio	Lectura del contador en la instrucción siguiente al encendido del LED objetivo	4.3
Botón correcto termina la ronda	Comparación del valor de los botones con la máscara del LED objetivo	5.7

Requisito del enunciado	Cómo se cumple	Sección
Tiempo en décimas en los displays	⌊(ciclos + 1.250.000) / 2.500.000⌋ y conversión a dos dígitos decimales	4.4
Diez respuestas correctas	Contador de rondas s1, constante NUM_ROUNDS = 10	5.2
Promedio en la misma unidad	Σ ciclos / 25.000.000, redondeado, en décimas	4.5
Error: indicar y repetir la ronda	EE + 3 parpadeos; s1 y tp no se modifican	5.7
Contador de ciclos accesible	Registro de 32 bits en 0x8000_000C	3.2
Frecuencia y conversión explícitas	25 MHz · 40 ns por ciclo · 2.500.000 ciclos por décima	4
Assembler propio game.s → game.hex	tools/rvasm.py, dos pasadas, 14 pruebas automáticas	6
README con síntesis y regeneración del hex	make, make hex y rvasm.py documentados	8.1

2. DISEÑO GENERAL Y ARQUITECTURA

2.1. Decisión de diseño principal: hardware mínimo, lógica en software

La alternativa más directa sería construir el juego como hardware dedicado (temporizadores, generador aleatorio y conversor a decimal en Verilog). Se descartó por dos razones: el enunciado pide que el juego sea un programa en assembly, y el SoC base ya ocupa el 83 % de las celdas lógicas de la HX1K (1064 de 1280), lo que deja muy poco espacio.

Por eso al hardware se le agregó una sola pieza funcional nueva: un **contador de ciclos de 32 bits** que el programa puede leer. Todo lo demás (los 3 segundos, la medición del tiempo, la conversión a décimas, el promedio y la selección pseudoaleatoria) se resuelve en software a partir de ese contador.

Necesidad del juego	Dónde se resuelve	Cómo
Esperar 3 s	Software	Restar dos lecturas del contador hasta llegar a 75.000.000 ciclos
Medir el tiempo de reacción	Software	Diferencia entre la lectura al encender el LED y la lectura al detectar el botón
Convertir a décimas	Software	División por restas sucesivas (no hay instrucción de división)
Mostrar en decimal	Software	Conversión a dos dígitos BCD con sumas (no hay shifts)
Número pseudoaleatorio	Software	Generador congruencial + entropía del contador
Base de tiempo	Hardware nuevo	Contador de ciclos en 0x8000_000C
Botones confiables	Hardware nuevo + software	Sincronizador de 2 flip-flops + antirrebote por software

2.2. Diagrama de bloques

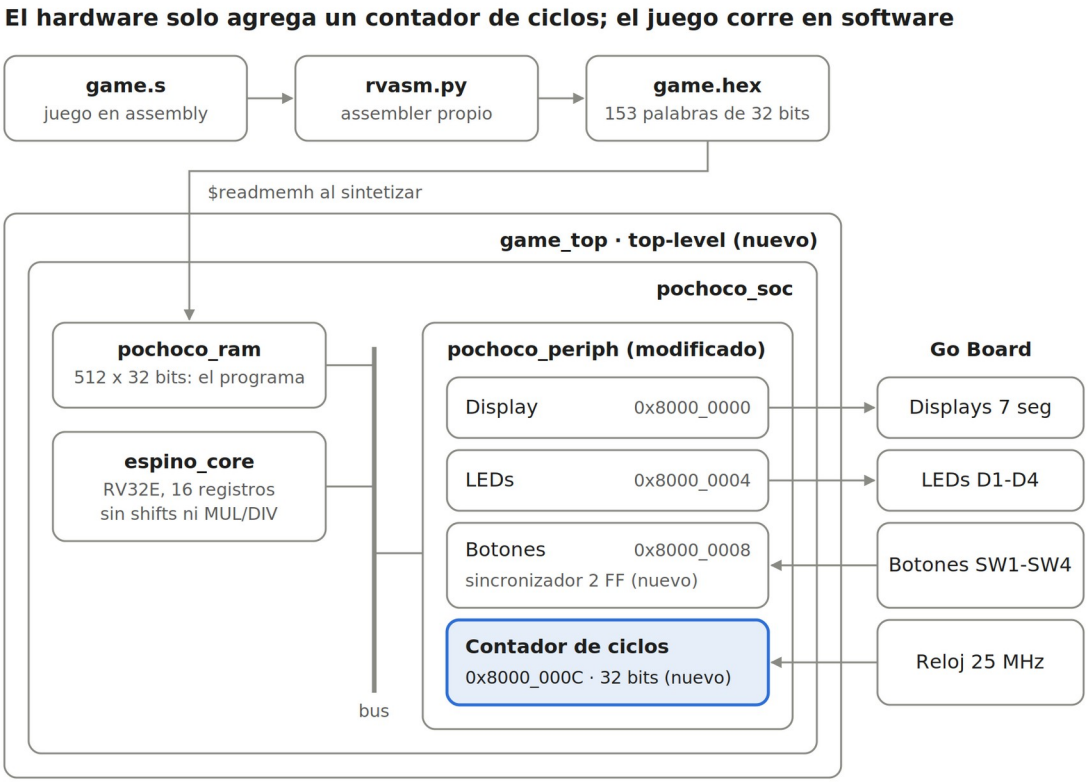


Figura 1. Diagrama de bloques del sistema. El único bloque funcional nuevo es el contador de ciclos.

El assembler genera `game.hex`, que la síntesis incrusta en la RAM del SoC. En la FPGA, el procesador ejecuta el juego y accede a todos los periféricos por el bus. El único bloque funcional nuevo es el contador de ciclos, alimentado por el reloj de 25 MHz de la placa.

2.3. Jerarquía de módulos

- **game_top:** top-level que se sintetiza (nuevo)
  - **pochoco_soc:** el SoC, instanciado con MemFile = "sw/game.hex"
    - **espino_core:** procesador RV32E (sin cambios)
    - **pochoco_ram:** 512 palabras de 32 bits; contiene el programa del juego
    - **pochoco_periph:** LEDs, botones, displays y **contador de ciclos** (modificado)
    - **pochoco_spi_slave:** SPI (sin uso en el juego)

2.4. Mapa de memoria

El programa ve los periféricos como direcciones de memoria a partir de 0x8000_0000. El programa se carga desde la dirección 0 de la RAM.

Dirección	Acceso	Contenido
0x0000_0000 – 0x0000_07FF	Lectura/escritura	RAM: programa del juego (153 de 512 palabras, 30 %)
0x8000_0000	Escritura	Displays: nibble alto = dígito izquierdo, nibble bajo = dígito derecho
0x8000_0004	Escritura	LEDs, bits 3..0
0x8000_0008	Lectura	Botones sincronizados, bits 3..0 (1 = presionado)
0x8000_000C	Lectura	Contador de ciclos de 32 bits (nuevo)

3. MODIFICACIONES AL RTL

Se hicieron tres cambios al hardware. El núcleo Espino Core, la RAM y el SPI no se modificaron.

3.1. Top-level game_top.v (nuevo)

Es el módulo que se sintetiza y se programa en la FPGA. Instancia `pochoco_soc` y fija la imagen de memoria del juego con el parámetro `MemFile = "sw/game.hex"`. Sus puertos (reloj, LEDs, botones, displays y SPI) pasan directo al SoC. La ruta del hex es relativa al directorio donde corre Yosys, la raíz del repositorio. Durante la síntesis, `$readmemh` incrusta el programa en la memoria interna (BRAM) de la FPGA; por eso, cada vez que cambia `game.hex` hay que volver a sintetizar.

3.2. Contador de ciclos (en pochoco_periph.v)

Es un registro de 32 bits que suma 1 en cada flanco de subida del reloj y se reinicia con el reset del SoC. El programa lo lee con `lw` en 0x8000_000C, agregado como un caso más del multiplexor de lectura de periféricos.

```
reg [31:0] cycle_q;
always @(posedge clk_i or negedge rst_ni) begin
  if (!rst_ni) cycle_q <= 32'b0;
  else         cycle_q <= cycle_q + 32'd1;
end
// en el multiplexor de lectura:
6'd3: rdata_o <= cycle_q;   // 0x8000_000C
```

El contador nunca se reinicia durante el juego: el software trabaja solo con **diferencias** (fin − inicio) en aritmética sin signo. Así no hace falta lógica para borrarlo o detenerlo, y la resta sigue siendo correcta aunque el contador dé la vuelta entre las dos lecturas. Esto vale mientras el intervalo medido sea menor que el periodo del contador, 2^{32} / 25.000.000 = 171,8 s; ninguna medición del juego se acerca a ese valor (la mayor es la saturación de 9,9 s).

3.3. Sincronizador de botones (en pochoco_periph.v)

En el SoC original los botones llegaban directo al bus. Como son entradas asíncronas, un botón que cambia justo en el flanco del reloj puede dejar un flip-flop en estado metaestable. Se agregaron dos etapas de flip-flops (`btn_meta_q` → `btn_sync_q`) y el bus lee la segunda. El costo es una latencia de 2 ciclos (80 ns), despreciable frente a la resolución de una décima.

3.4. Costo en recursos

Diseño	Celdas lógicas (LC)	BRAM	Frecuencia máxima
SoC original	1064 / 1280 (83 %)	12 / 16	48 MHz
Con las modificaciones	1189 / 1280 (92 %)	12 / 16	45 MHz (PASS a 25 MHz)

Las modificaciones cuestan unas 125 celdas lógicas, principalmente por el sumador de 32 bits del contador. Este margen tan ajustado es la razón para no agregar un desplazador (shifter) al ALU, que habría costado bastante más.

4. TEMPORIZACIÓN Y CONVERSIÓN DE UNIDADES

Todos los tiempos del juego se derivan de una sola constante: la frecuencia del reloj de la Go Board, **25 MHz**. Un ciclo dura **40 ns** y una décima de segundo son **2.500.000 ciclos**.

4.1. Constantes de tiempo

En `game.s` solo `CLK_HZ` depende de la placa; el resto se calcula a partir de ella con expresiones que resuelve el assembler. Cambiar la frecuencia (por ejemplo, para simular) actualiza todos los tiempos a la vez.

Constante	Expresión en game.s	Valor (ciclos)	Tiempo
CLK_HZ	25000000	25.000.000	1 s
TENTH	CLK_HZ / 10	2.500.000	0,1 s
HALF_TENTH	TENTH / 2	1.250.000	0,05 s (redondeo)
ALL_ON_TIME	30 * TENTH	75.000.000	3,0 s
MAX_CYCLES	MAX_TENTHS * TENTH	247.500.000	9,9 s (saturación)
DEBOUNCE	TENTH / 5	500.000	20 ms
BLINK_TIME	TENTH * 3 / 2	3.750.000	150 ms
IDLE_STEP	2 * TENTH	5.000.000	200 ms

4.2. Espera de tres segundos

La subrutina `delay` lee el contador al entrar y luego lo vuelve a leer en un ciclo hasta que la diferencia alcanza el número de ciclos pedido (`bltu` compara sin signo). Como se basa en el contador y no en contar instrucciones, la duración es exacta sin importar cuántos ciclos tome cada instrucción del pipeline.

Después de los 3,0 s el programa llama a `wait_release`, que exige 20 ms seguidos sin botones presionados. Por eso la fase con los cuatro LEDs dura en la práctica 3,02 s, lo que cumple el requisito de "al menos tres segundos". Si el jugador mantiene un botón apretado, la fase se alarga hasta que lo suelte: así no se puede ganar una ronda con un botón ya presionado.

4.3. Medición del tiempo de reacción

El enunciado pide iniciar la medición "en el mismo instante lógico en que comienza la ronda". Se interpretó el comienzo de la ronda que se mide como el momento en que se enciende el LED objetivo: es el primer instante en que el jugador tiene la información necesaria para reaccionar, mientras que la fase de 3 s es una preparación idéntica en todas las rondas. Si se midiera desde el encendido de los cuatro LEDs, todos los tiempos incluirían los mismos 3,02 s fijos, que no dicen nada del reflejo del jugador. Aun así, el cambio a la otra interpretación es de una sola línea: mover la lectura del contador (`lw a4`) antes de la fase inicial.

Con esta interpretación, el "instante lógico" queda definido por una instrucción concreta: la lectura del contador (`lw a4, CYC_OFF(s0)`) es la **inmediatamente siguiente** al `sw` que enciende el LED. Mientras espera, el programa lee el contador **antes** que los botones en cada vuelta del ciclo; cuando detecta una pulsación, la última lectura del contador corresponde a ese instante.

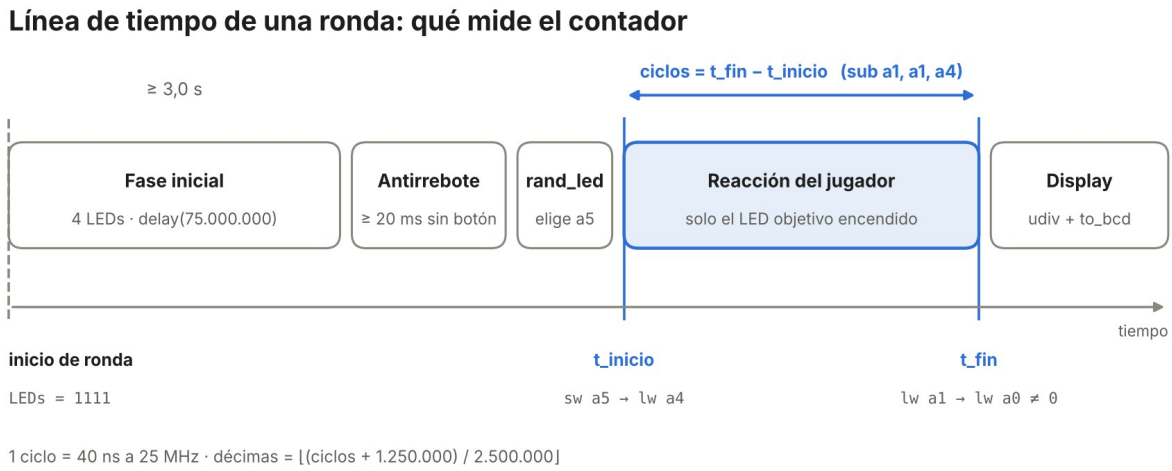


Figura 2. Línea de tiempo de una ronda. El contador se lee al encender el LED objetivo y al detectar el botón.

```
sw   a5, LED_OFF(s0)        # se enciende el LED objetivo
lw   a4, CYC_OFF(s0)        # t_inicio
wait_press:
lw   a1, CYC_OFF(s0)        # t_actual
lw   a0, BTN_OFF(s0)        # botones
beqz a0, wait_press
sub  a1, a1, a4             # ciclos = t_fin - t_inicio
```

El error de este método está acotado por tres latencias pequeñas: un ciclo entre el `sw` y el `lw`, los 2 ciclos del sincronizador de botones y la duración de una vuelta del ciclo `wait_press` (del orden de 10 ciclos). En total son unos pocos ciclos de reloj (menos de 1 µs), del orden de cien mil veces menor que la décima de segundo que se muestra.

4.4. Conversión de ciclos a décimas

Se redondea al entero más cercano sumando media décima antes de dividir:

$$\textbf{décimas} = \lfloor (\text{ciclos} + 1.250.000) / 2.500.000 \rfloor$$

Ejemplo: una reacción de 0,47 s son 11.750.000 ciclos; (11.750.000 + 1.250.000) / 2.500.000 = 5,2, que se trunca a 5 y se muestra como **05**. Como el procesador no tiene instrucción de división, la subrutina `udiv` resta el divisor repetidamente y cuenta las restas. El cociente nunca supera 99, así que son a lo más 99 iteraciones (unos pocos microsegundos).

El display tiene solo dos dígitos, por lo que cada ronda **se satura en 9,9 s**: si los ciclos superan 247.500.000 se reemplazan por ese valor. Esto además evita que la suma de las diez rondas desborde 32 bits: su máximo es 2.475.000.000, menor que $2^{32} - 1 = 4.294.967.295$.

4.5. Cálculo del promedio

El programa acumula en el registro `tp` los **ciclos** de cada respuesta correcta, no las décimas ya redondeadas. Al final aplica una sola división:

$$\textbf{promedio [décimas]} = (\Sigma \text{ ciclos} / 10) / 2.500.000 = \lfloor (\Sigma \text{ ciclos} + 12.500.000) / 25.000.000 \rfloor$$

Promediar los ciclos evita acumular diez errores de redondeo. Además, la división por 10 y la división por una décima se combinan en una sola división por 25.000.000 (exactamente un segundo de reloj), con su propio redondeo.

Una consecuencia que conviene tener presente al verificar el resultado en la placa: el promedio mostrado puede diferir en una décima del promedio de los diez valores que se vieron en el display, porque esos valores ya venían redondeados. Por ejemplo, con cinco reacciones de 0,15 s (se muestran como 02) y cinco de 0,14 s (se muestran como 01), el promedio de lo mostrado es 1,5, pero el promedio real es 1,45 décimas y el display muestra **01**. El valor de la placa es el más exacto.

Límite de rondas. Gracias a la saturación de 9,9 s por ronda, la suma de ciclos no desborda 32 bits mientras NUM_ROUNDS ≤ 17 ($17 \times 247.500.000 = 4.207.500.000 < 2^{32}$). Para más rondas habría que reducir MAX_TENTHS o acumular décimas en lugar de ciclos.

5. PROGRAMA EN ASSEMBLY

5.1. Restricciones del procesador y cómo se resolvieron

Limitación del Espino Core	Operación que la necesitaba	Solución usada
Shifts deshabilitados (se ejecutan como ADD)	Multiplicar por 16 para armar los dos dígitos	Cuatro add x, x, x (cada uno duplica)
Shifts deshabilitados	Extraer los bits 17 y 16 del número aleatorio	Máscaras con and y lui (ej. lui t1, 0x10 = bit 16)
Sin multiplicación (RV32E base)	Generador congruencial 5·x	x + x → 2x, + 2x → 4x, + x → 5x
Sin división	Ciclos → décimas, promedio, decenas	Restas sucesivas (udiv, to_bcd)
Solo 16 registros (x0–x15)	Variables del juego	Convención fija de registros (5.2) y subrutinas sin anidar
Sin pila necesaria	Llamadas a subrutinas	Todas las subrutinas son "hoja": no llaman a otras, así ra nunca se pisa

5.2. Uso de registros

Registro	Nombre	Uso
x8	s0	Base de periféricos 0x8000_0000 (fija todo el programa)
x9	s1	Rondas correctas acumuladas (0 a 10)
x4	tp	Suma de ciclos de las rondas correctas (para el promedio)
x3	gp	Estado del generador pseudoaleatorio
x15	a5	Máscara del LED objetivo (1, 2, 4 u 8)
x14	a4	Ciclo de inicio de la medición
x12	a2	Contador auxiliar del programa principal (carrusel, parpadeos)
x10, x11	a0, a1	Argumentos y resultado de las subrutinas
x5–x7, x13	t0–t2, a3	Temporales; las subrutinas pueden modificarlos
x1	ra	Dirección de retorno (jal ra, … / ret)

Los registros `gp` y `tp` tienen un uso especial en la convención de RISC-V, pero este programa no usa sistema operativo ni hilos, así que se aprovechan como variables globales.

5.3. Flujo del programa

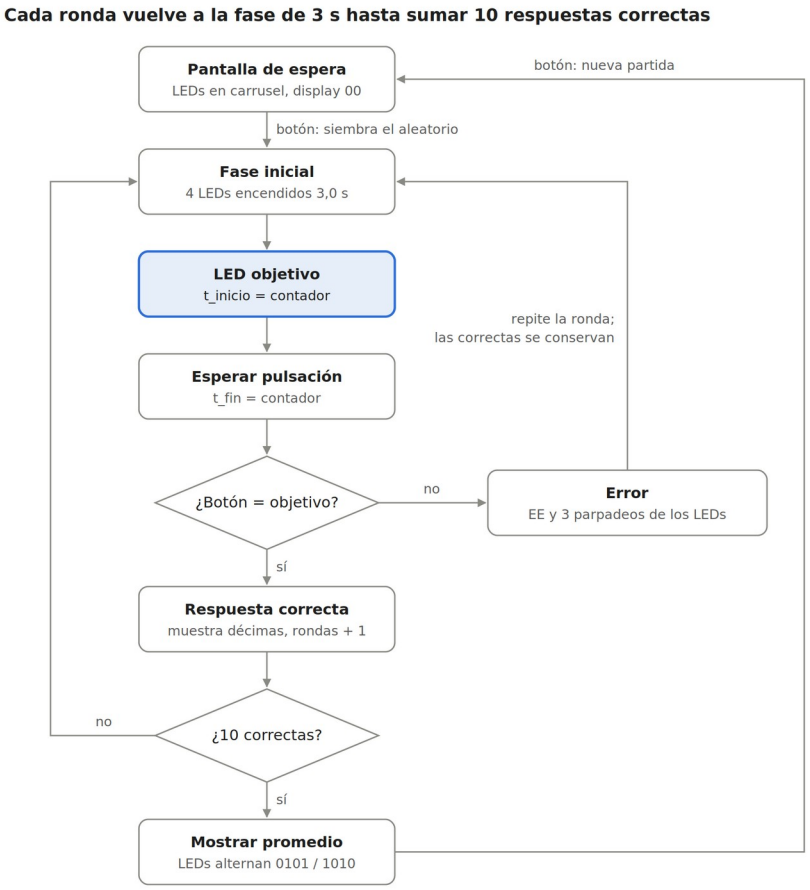


Figura 3. Diagrama de flujo del juego.

Un error devuelve a la fase de 3 s sin tocar el conteo de rondas correctas. La medición empieza al encender el LED objetivo.

5.4. Subrutinas

Subrutina	Entrada	Salida	Qué hace
delay	a0 = ciclos	(ninguna)	Espera a0 ciclos leyendo el contador
wait_or_press	a0 = ciclos	a0 = botones (0 si se agotó el tiempo)	Espera hasta a0 ciclos o hasta una pulsación; anima las pantallas de espera y final
wait_release	(ninguna)	(ninguna)	Antirrebote: retorna tras 20 ms seguidos sin botones
rand_led	gp	a5 = 1, 2, 4 u 8	Avanza el generador y elige el LED objetivo

Subrutina	Entrada	Salida	Qué hace
udiv	a0 dividendo, a1 divisor	a0 = cociente	División entera sin signo por restas
to_bcd	a0 = 0..99	a0 = decenas·16 + unidades	Convierte a dos dígitos decimales para el display

5.5. Generación pseudoaleatoria

El estado `gp` combina un generador congruencial lineal con entropía humana:

1. **Semilla:** en la pantalla de espera inicial, el ciclo exacto en que el jugador presiona un botón se combina con la semilla (`xor gp, gp, contador`). Ese ciclo cambia de una partida a otra, porque a 25 MHz una diferencia de 40 ns ya es otro valor.
2. **Mezcla por ronda:** cada tiempo de reacción medido se suma a `gp`.
3. **Avance:** `rand_led` calcula gp = 5·gp + 12345 + contador. La parte congruencial (multiplicador 5, con 5 − 1 divisible por 4, e incremento impar) tiene periodo completo de 2^{32} por el teorema de Hull–Dobell; la suma del contador agrega entropía extra en cada ronda.
4. **Selección:** se toman los bits 17 y 16 de `gp`. En un generador módulo 2^{32} los bits bajos tienen periodo muy corto (el bit 0 alterna), por eso se usan bits altos. Sin shifts, cada bit se prueba con una máscara: si el bit 16 vale 1, el LED se duplica; si el bit 17 vale 1, se cuadruplica.

La pantalla de espera inicial existe precisamente para esto. Sin ella, la primera ronda sería siempre la misma tras cada reset, porque el contador parte en 0 y el programa siempre tarda lo mismo en llegar a la primera ronda. Al repetirse una ronda por error, `rand_led` se vuelve a llamar, así que el LED objetivo del reintento también es nuevo.

5.6. Antirrebote

Los pulsadores mecánicos rebotan durante algunos milisegundos al soltarse. `wait_release` reinicia una ventana de tiempo cada vez que ve un botón presionado y solo retorna cuando pasan 20 ms seguidos sin pulsaciones. Se llama después de cada respuesta y después de un error, de modo que un rebote nunca se cuenta como una nueva pulsación.

5.7. Respuesta correcta y manejo de error

Al detectar una pulsación, el programa compara el valor leído de los botones con la máscara del LED objetivo (`bne a0, a5, wrong_button`). Si coinciden, la ronda termina: se acumulan los ciclos, se muestra el tiempo y se incrementa `s1`. Si el valor es distinto (otro botón, o varios a la vez), el programa:

1. Muestra **EE** en el display.
2. Espera a que se suelten los botones.
3. Hace parpadear los cuatro LEDs 3 veces (150 ms encendidos, 150 ms apagados).

4. Vuelve a `round_start`: repite la ronda completa, incluida la fase de 3 s.

Los registros `s1` (rondas correctas) y `tp` (suma de ciclos) no se tocan, así que las rondas correctas acumuladas se conservan y el tiempo de la ronda fallida no entra al promedio.

5.8. Instrucciones utilizadas

El juego usa 16 instrucciones base de RV32I: `lui`, `addi`, `xori`, `add`, `sub`, `and`, `xor`, `lw`, `sw`, `jal`, `jalr`, `beq`, `bne`, `blt`, `bltu`, `bgeu`. También usa 6 pseudo-instrucciones, que el assembler expande a esas instrucciones base: `li`, `mv`, `j`, `ret`, `beqz` y `bnez`. No usa ningún shift. Esta lista se verificó decodificando las 153 palabras de `game.hex`. El listado completo con dirección, código de máquina y fuente está en el Anexo A.

6. ASSEMBLER

El assembler (`tools/rvasm.py`, Python 3, sin dependencias externas) recibe `game.s` y genera `game.hex`: una palabra de 32 bits en hexadecimal (8 dígitos) por línea, el formato que lee `$readmemh`. No invoca ningún assembler RISC-V externo.

```
python3 tools/rvasm.py sw/game.s -o sw/game.hex [-l sw/game.lst] [-D NOMBRE=valor]
```

La opción `-l` genera un listado con dirección, código máquina y línea fuente. La opción `-D` redefine una constante `.equ` desde la línea de comandos, con prioridad sobre el fuente; se usa en la simulación (`-D CLK_HZ=20000`).

6.1. Sintaxis aceptada

- **Una instrucción por línea**, con operandos separados por comas: `add a0, a1, a2`.
- **Comentarios:** `#`, `;` o `//` hasta el fin de la línea.
- **Etiquetas:** `nombre:` al inicio de la línea, solas o seguidas de una instrucción.
- **Registros:** x0–x15 o nombres ABI (zero, ra, sp, gp, tp, t0–t2, s0/fp, s1, a0–a5).
- **Memoria:** offset(registro), por ejemplo `lw a0, 8(s0)` o `sw t0, LED_OFF(s0)`.
- **Inmediatos:** decimal, hexadecimal (0x), binario (0b) o expresiones con constantes, etiquetas y los operadores + - * / % & | ^ ~ << >> con paréntesis. También acepta %hi(x) y %lo(x).
- **Directivas:** `.equ` / `.set` (constantes), `.word` (datos de 32 bits). `.text`, `.global`, `.globl`, `.section`, `.data` y `.align` se aceptan y se ignoran, porque todo se carga en RAM desde la dirección 0.

6.2. Instrucciones soportadas

Formato	Instrucciones
R	add sub and or xor slt sltu sll srl sra
I (ALU)	addi andi ori xori slti sltiu slli srli srai
I (carga)	lw lh lb lhu lbu jalr
S	sw sh sb
B	beq bne blt bge bltu bgeu
U	lui auipc
J	jal
Pseudo	nop li la mv not neg seqz snez j jr call ret beqz bnez bltz bgez blez bgtz bgt ble bgtu bleu

Los shifts se codifican correctamente según la especificación, pero el assembler emite un aviso: el ALU del Espino Core los ejecuta como suma.

6.3. Tratamiento de etiquetas: dos pasadas

Un salto hacia adelante (por ejemplo, `bne a0, a5, wrong_button`) usa una etiqueta que todavía no se ha leído. Por eso el assembler recorre el archivo dos veces:

1. **Primera pasada:** asigna a cada línea su dirección, registra cada etiqueta con su dirección y evalúa las constantes `.equ`. Para avanzar la dirección necesita saber cuántas palabras ocupa cada instrucción; todas ocupan una, salvo `li` y `la`.
2. **Segunda pasada:** con todas las etiquetas conocidas, codifica cada instrucción. En los saltos (`jal`, branches) el inmediato es relativo: offset = dirección de la etiqueta − dirección de la instrucción.

El caso delicado es `li`, que ocupa 1 o 2 palabras según su valor. En la primera pasada se decide el tamaño: 1 palabra si el valor cabe en 12 bits con signo (`addi rd, x0, v`) o si sus 12 bits bajos son cero (`lui` solo); 2 palabras en otro caso. Si el valor depende de un símbolo aún desconocido, se reservan 2 palabras. La segunda pasada verifica que el tamaño no cambió; si cambiara, todas las direcciones posteriores estarían corridas.

6.4. Tratamiento de inmediatos: li con 32 bits

Un valor de 32 bits se divide en `lui` (20 bits altos) + `addi` (12 bits bajos). La dificultad es que `addi` extiende el signo de su inmediato: si el bit 11 de la parte baja es 1, `addi` resta en vez de sumar. El assembler lo compensa sumando 1 a la parte alta:

hi = ⌊ (v + 0x800) / 2^{12} ⌋ mod 2^{20} lo = v − hi · 2^{12}, con −2048 ≤ lo ≤ 2047

Ejemplo con `li a0, ALL_ON_TIME` (75.000.000 = 0x047868C0). La parte baja 0x8C0 tiene el bit 11 en 1, así que vale −1856 como número con signo. La parte alta se redondea hacia arriba a 0x4787:

Instrucción	Código	Efecto
lui a0, 0x4787	04787537	a0 = 0x0478_7000 = 75.001.856
addi a0, a0, -1856	8c050513	a0 = 75.001.856 − 1856 = 75.000.000

6.5. Ejemplo de traducción a código máquina

Extracto del listado `sw/game.lst` (dirección · código · fuente), en el instante en que empieza la medición:

```
0000006c   00f42223    sw   a5, LED_OFF(s0)    # se enciende el LED objetivo
00000070   00c42703    lw   a4, CYC_OFF(s0)    # inicio de la medición
00000074   00c42583    lw   a1, CYC_OFF(s0)    # wait_press:
00000078   00842503    lw   a0, BTN_OFF(s0)
0000007c   fe050ce3    beqz a0, wait_press
```

lw a4, CYC_OFF(s0) (formato I). Con CYC_OFF = 12, s0 = x8 y a4 = x14:

Campo	imm[11:0]	rs1	funct3	rd	opcode
Valor	12	x8	LW	x14	LOAD
Bits	000000001100	01000	010	01110	0000011

Al concatenar: 0000 0000 1100 0100 0010 0111 0000 0011 = **0x00C42703**.

beqz a0, wait_press (formato B). Es la pseudo-instrucción `beq a0, x0, wait_press`. La instrucción está en 0x7C y wait_press en 0x74, así que el offset es −8 = 1 1111 1111 1000 en 13 bits (complemento a dos). El formato B reparte esos bits de forma no contigua:

Campo	imm [12]	imm [10:5]	rs2	rs1	funct3	imm [4:1]	imm [11]	opcode
Bits	1	111111	00000	01010	000	1100	1	1100011

Al concatenar: 1111 1110 0000 0101 0000 1100 1110 0011 = **0xFE050CE3**. El bit 0 del offset no se guarda, porque los destinos siempre son pares.

6.6. Validación de errores

El assembler informa cada error con archivo, número de línea y la línea fuente, y **no genera el .hex** si hay alguno:

Error detectado	Ejemplo	Mensaje
Registro inexistente en RV32E	add x16, x1, x2	registro 'x16' no existe en RV32E (solo x0..x15)
Inmediato fuera de rango	addi a0, a0, 2048	inmediato de 12 bits fuera de rango
Salto demasiado lejano	beq a más de ±4 KiB	salto condicional fuera de rango
Etiqueta indefinida	beq x0, x0, nada	simbolo no definido: 'nada'
Etiqueta duplicada	a: dos veces	etiqueta/simbolo duplicado
Instrucción desconocida	foo a0, a1	instruccion desconocida
Operandos incorrectos	add a0, a1	'add' espera 3 operandos
Operando de memoria mal escrito	lw a0, a1	se espera offset(reg)
Programa más grande que la RAM	más de 512 palabras	el programa ocupa N palabras y la RAM tiene 512

6.7. Pruebas

`tools/test_rvasm.py` contiene 14 pruebas automáticas (`make test`), todas aprobadas:

- **Contra referencia GNU:** ensambla los tres ejemplos del repositorio (blink.s, 7seg.s, buttons_leds.s) y compara palabra por palabra con los .hex oficiales, generados con el toolchain GNU de RISC-V. Coinciden exactamente.
- **Codificación:** 25 instrucciones de todos los formatos contra su código esperado, verificado además con un desensamblador independiente.
- **Etiquetas y pseudo-instrucciones:** saltos hacia adelante y atrás, `li` corto y largo (incluido el caso con compensación de signo), `.equ` con expresiones y `-D`.
- **Errores:** cada caso de la tabla 6.6, comprobando también el número de línea.

Salida de `make test`:

```
test_casos (TestCodificacion) ... ok
test_branch_fuera_de_alcance (TestErrores) ... ok
test_etiqueta_indefinida_y_duplicada (TestErrores) ... ok
test_inmediato_fuera_de_rango (TestErrores) ... ok
test_mnemonico_y_operandos (TestErrores) ... ok
test_registro_rv32e (TestErrores) ... ok
test_tamano_ram (TestErrores) ... ok
test_branch_adelante_y_atras (TestEtiquetasYPseudo) ... ok
test_equ_expresiones_y_define (TestEtiquetasYPseudo) ... ok
test_etiqueta_y_instruccion_misma_linea (TestEtiquetasYPseudo) ... ok
test_jal_y_j (TestEtiquetasYPseudo) ... ok
test_li_corto_y_largo (TestEtiquetasYPseudo) ... ok
test_word (TestEtiquetasYPseudo) ... ok
test_ejemplos_del_repo (TestReferencia) ... ok
----------------------------------------------------------------------
Ran 14 tests in 0.011s

OK
```

7. SIMULACIÓN Y VERIFICACIÓN

El juego completo se simuló con Icarus Verilog (`make sim`) antes de llevarlo a la placa: el SoC con el RTL modificado ejecutando `game.s`, y un testbench (`sim/tb_game.v`) que hace de jugador.

7.1. Reloj ficticio para simular

Con el reloj real, una partida dura cientos de millones de ciclos, demasiados para simular. Como todas las constantes se derivan de `CLK_HZ` (sección 4.1), basta ensamblar con `-D CLK_HZ=20000`: una décima pasa a ser 2000 ciclos y la fase de 3 s, 60.000 ciclos. El programa simulado es el mismo de la placa; solo cambian los valores numéricos de las constantes.

7.2. Partida simulada

El testbench presiona un botón en la pantalla de espera y juega 10 rondas. En cada una detecta el LED objetivo, espera el tiempo de reacción programado y presiona el botón correcto. En la ronda 3 presiona primero un botón incorrecto. Después de cada pulsación compara el display con el valor esperado.

Ronda	Reacción programada	Display esperado	Resultado
1	0,3 s	03	Correcto
2	0,5 s	05	Correcto
3	botón incorrecto	EE, se repite la ronda	Correcto
3 (reintento)	1,2 s	12	Correcto
4	0,7 s	07	Correcto
5	12,0 s	99 (saturado)	Correcto
6	0,4 s	04	Correcto
7	0,9 s	09	Correcto
8	1,5 s	15	Correcto
9	0,6 s	06	Correcto
10	1,1 s	11	Correcto
Promedio	(3+5+12+7+99+4+9+15+6+11) / 10 = 17,1	17	Correcto

El testbench también verificó en cada ronda que la fase de cuatro LEDs durara al menos 30 décimas. En las 10 rondas normales midió entre 60.487 y 60.491 ciclos, frente a un mínimo de 60.000; en el reintento midió 78.646, porque incluye el último parpadeo del error. La diferencia corresponde a los 20 ms de antirrebote más algunas instrucciones. Resultado final: **todas las pruebas pasaron**.

7.3. Simulación post-síntesis

Además se simuló el circuito **ya sintetizado por Yosys**, es decir, la red de celdas de la iCE40 que se graba en la placa, no el Verilog original. En esa simulación el diseño hizo el carrusel de la pantalla de espera, respondió a un botón con la fase de 4 LEDs y encendió un LED objetivo tras 60.490 ciclos. Esto confirma que la síntesis no altera el comportamiento.

8. IMPLEMENTACIÓN EN FPGA

8.1. Flujo de compilación

Todo el flujo se ejecuta con `make` desde la raíz del repositorio. En Windows con Git Bash se agrega `PYTHON=python`.

1. `tools/rvasm.py` ensambla `sw/game.s` → `sw/game.hex` (y el listado `sw/game.lst`).
2. **Yosys** sintetiza el RTL con `game_top` como top-level; `$readmemh` incrusta `game.hex` en las BRAM.
3. **nextpnr-ice40** ubica y conecta el diseño en la HX1K (VQ100), con restricción de reloj de 25 MHz.
4. **icepack** genera el bitstream `game_top.bin`.
5. **iceprog** lo graba en la flash de la Go Board.

Comando	Qué hace
make hex	Solo ensambla game.s → game.hex
make bin	Ensambla, sintetiza y genera el bitstream, sin programar
make	Todo lo anterior y además programa la Go Board
make sim	Simula una partida completa
make test	Corre las pruebas del assembler
make clean	Borra los archivos generados para recompilar desde cero

El Makefile declara que `game.hex` depende de `game.s` y del assembler, y que la síntesis depende de `game.hex`. Así, al editar una constante en `game.s` basta con correr `make`: se regenera el hex, se resintetiza y se reprograma la placa.

8.2. Resultados de síntesis

Métrica	Valor
Dispositivo	Lattice iCE40 HX1K, package VQ100
Celdas lógicas	1189 / 1280 (92 %)
Bloques de RAM (BRAM)	12 / 16 (75 %)
Frecuencia máxima	45,04 MHz (la placa opera a 25 MHz: PASS)
Tamaño del programa	153 palabras = 612 bytes, de 2048 disponibles
Tamaño del bitstream	32.220 bytes
Programación	iceprog: VERIFY OK

El place & route terminó con 0 errores y 2 advertencias. Ambas corresponden a los pines del UART definidos en `goboard.pcf`, que este diseño no usa.

8.3. Interfaz

Elemento	Uso en el juego
LEDs D1–D4	Carrusel de espera, fase de 3 s (los cuatro), LED objetivo, parpadeo de error, alternancia final
Botones SW1–SW4	Cada botón corresponde al LED de su misma posición (SW1 ↔ D1, …, SW4 ↔ D4); cualquiera inicia la partida
Display de 7 segmentos	Tiempo de la ronda en décimas (00–99), EE ante error, promedio al final

El display muestra el tiempo en décimas de segundo con dos dígitos decimales, sin coma: 05 = 0,5 s y 12 = 1,2 s.

8.4. Secuencia de uso

1. **Espera:** al encender, los LEDs giran en carrusel y el display muestra 00. Cualquier botón inicia la partida y siembra el generador aleatorio.
2. **Fase inicial:** los cuatro LEDs se encienden durante 3 segundos.
3. **Ronda:** queda encendido solo el LED objetivo y empieza la medición.
4. **Respuesta correcta:** el display muestra el tiempo en décimas y empieza la siguiente ronda. El tiempo queda visible durante la fase de 3 s.
5. **Respuesta incorrecta:** el display muestra EE, los LEDs parpadean 3 veces y se repite la ronda; las rondas correctas se conservan.
6. **Final:** tras 10 respuestas correctas el display muestra el promedio y los LEDs alternan 0101 / 1010. Un botón vuelve a la pantalla de espera para una nueva partida.

8.5. Demostración en la placa

A continuación se documenta una partida completa en la Go Board, con cada etapa de la secuencia anterior:



Figura 4a. Pantalla de espera: carrusel, display 00

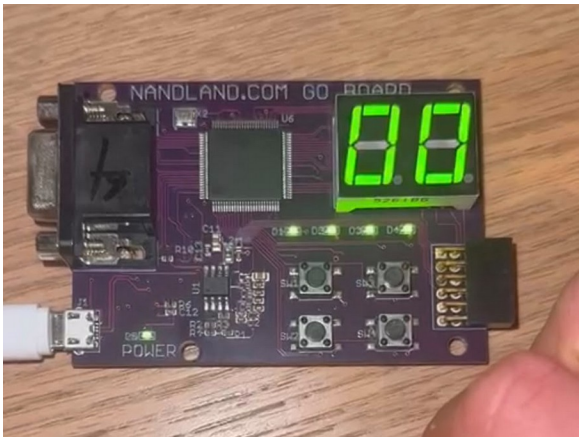


Figura 4b. Fase inicial: los cuatro LEDs durante 3 s

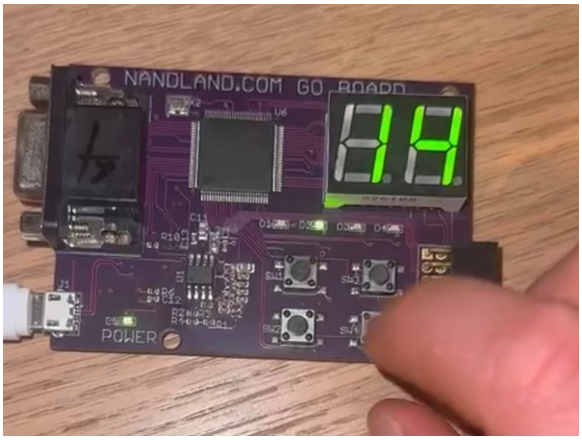


Figura 4c. LED objetivo D2; el display aún muestra el tiempo anterior (14)

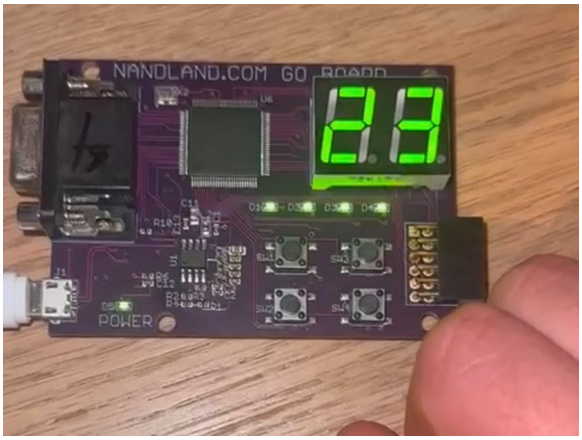


Figura 4d. Ronda correcta: 23 = 2,3 s, visible durante la fase de 3 s siguiente

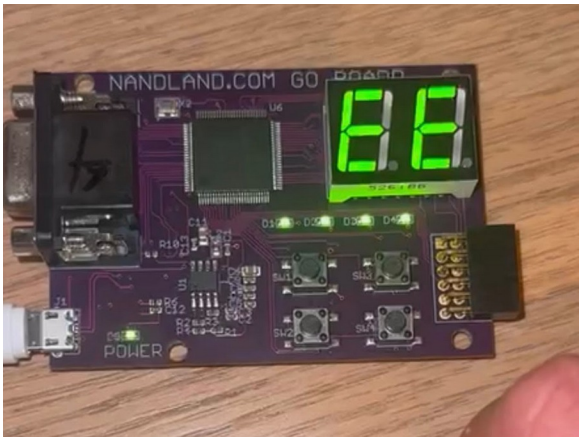


Figura 4e. Botón incorrecto: EE y parpadeo de los LEDs

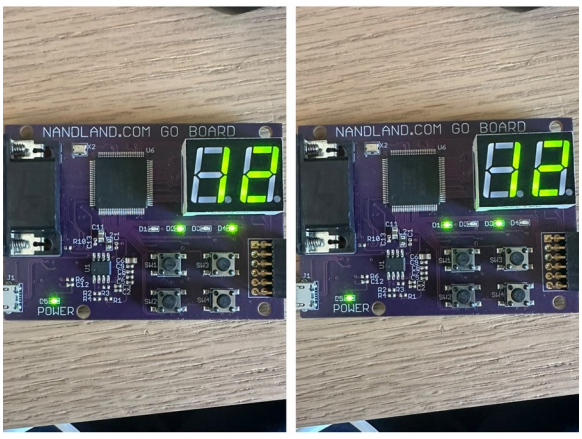


Figura 4f. Promedio final 12 = 1,2 s; LEDs alternan 1010 / 0101

En la Figura 4d, los cuatro LEDs están encendidos porque ya comenzó la fase de 3 s de la ronda siguiente; el tiempo de la ronda correcta permanece en el display durante esa fase. En la Figura 4f se muestran dos instantes de la pantalla final para evidenciar la alternancia de los LEDs mientras el display mantiene el promedio.

En todos los casos, el comportamiento observado en la placa coincidió con el verificado en la simulación (sección 7): la fase inicial dura 3 s, el tiempo mostrado corresponde a la reacción, un botón incorrecto muestra EE sin perder las rondas acumuladas y, tras diez respuestas correctas, se muestra el promedio. Esto confirma que el diseño funciona correctamente sobre la FPGA.

8.6. Cambio de una constante durante la evaluación

Para cambiar el comportamiento del juego se edita una constante en `sw/game.s` y se corre `make`. La tabla indica también el rango en que cada cambio es seguro:

Cambio pedido	Línea en game.s	Rango seguro
5 rondas en vez de 10	.equ NUM_ROUNDS, 5	1 a 17 (sección 4.5)
Fase inicial de 4 s	.equ ALL_ON_TIME, 40 * TENTH	Hasta ~170 s (vuelta del contador)
Antirrebote de 50 ms	.equ DEBOUNCE, TENTH / 2	Cualquier valor positivo
Más parpadeos de error	.equ ERR_BLINKS, 5	1 o más
Saturación más baja (ej. 5,0 s)	.equ MAX_TENTHS, 50	1 a 99 (dos dígitos)

También se puede cambiar sin editar el archivo: `python tools/rvasm.py sw/game.s -o sw/game.hex -D NUM_ROUNDS=5` y luego `make`. En ambos casos el cambio se verifica comparando el nuevo `game.hex` con el anterior (`git diff sw/game.hex`): solo cambian las palabras que cargan esa constante.

9. CONCLUSIONES

Agregar al SoC una sola pieza de hardware, un contador de ciclos de 32 bits, bastó para construir el juego completo en software. Los 3 segundos, la medición, la conversión a décimas, el promedio y la selección pseudoaleatoria salen todos de restar dos lecturas de ese contador. El costo fue de unas 125 celdas lógicas, en una FPGA que ya estaba al 83 %.

Las limitaciones del Espino Core obligaron a reconstruir operaciones que normalmente se dan por hechas. Sin shifts ni multiplicación, multiplicar por 16 son cuatro sumas y elegir un bit es aplicar una máscara. Sin división, convertir ciclos a décimas son restas sucesivas, y funciona bien porque el cociente nunca pasa de 99. Es la misma idea del proyecto 1, ahora en software: con un sumador se puede construir casi todo.

Trabajar con **diferencias** del contador, en vez de reiniciarlo, simplificó tanto el hardware como el software. La aritmética sin signo módulo 2^{32} hace que la resta sea correcta aunque el contador dé la vuelta, y ninguna parte del sistema necesita detenerlo ni borrarlo.

Derivar todas las constantes de tiempo de `CLK_HZ` hizo posible simular el programa real de la placa con un reloj ficticio de 20 kHz y verificar una partida completa en segundos. La misma estructura facilita cambiar cualquier constante durante la evaluación: se edita una línea y se corre `make`.

En el assembler, la parte con más detalle no fue codificar instrucciones, sino que las dos pasadas asignaran exactamente las mismas direcciones. Para eso cada pseudo-instrucción debe declarar su tamaño antes de conocer todos los símbolos, y la compensación de signo de `li` debe calcularse bien. Comparar la salida con la del toolchain GNU sobre los ejemplos del repositorio, palabra por palabra, dio la confianza para usarlo en el juego.

Las principales dificultades prácticas fueron de la interfaz física: los botones asíncronos (resueltos con el sincronizador de 2 flip-flops), sus rebotes (resueltos con el antirrebote por software) y la configuración del entorno para programar la placa desde Windows.

ANEXO A: LISTADO DEL PROGRAMA (sw/game.lst)

Listado generado por el assembler (`-l sw/game.lst`): dirección, código de máquina y línea fuente. Se omiten los comentarios de bloque; las instrucciones `li` de 32 bits ocupan dos palabras y la segunda aparece sin fuente.

```
                  _start:
00000000  80000437      li   s0, PERIPH  # s0 = 0x8000_0000
00000004  000011b7      li   gp, 0x1234  # semilla inicial (luego se mezcla con tiempos humanos)
00000008  23418193
                  new_game:
0000000c  00042023      sw   zero, SEG_OFF(s0)  # display "00"
00000010  00100613      li   a2, 1  # LED actual del carrusel
                  idle_loop:
00000014  00c42223      sw   a2, LED_OFF(s0)
00000018  004c5537      li   a0, IDLE_STEP
0000001c  b4050513
00000020  154000ef      jal  ra, wait_or_press  # a0 = botones (0 si se cumplio el tiempo)
00000024  00051c63      bnez a0, idle_pressed
00000028  00c60633      add  a2, a2, a2  # siguiente LED (x2, sin usar shifts)
0000002c  01000293      li   t0, 16
00000030  fe5612e3      bne  a2, t0, idle_loop
00000034  00100613      li   a2, 1
00000038  fddff06f      j    idle_loop
                  idle_pressed:
0000003c  00c42283      lw   t0, CYC_OFF(s0)
00000040  0051c1b3      xor  gp, gp, t0  # semilla <- semilla XOR contador de ciclos
00000044  158000ef      jal  ra, wait_release
00000048  00000493      li   s1, 0  # rondas correctas = 0
0000004c  00000213      li   tp, 0  # suma de ciclos = 0
                  round_start:
00000050  00f00293      li   t0, 15
00000054  00542223      sw   t0, LED_OFF(s0)  # 1) los cuatro LEDs encendidos
00000058  04787537      li   a0, ALL_ON_TIME
0000005c  8c050513
00000060  100000ef      jal  ra, delay  #    durante 3,0 s medidos con el contador
00000064  138000ef      jal  ra, wait_release  #    (si alguien mantiene un boton, se espera)
00000068  158000ef      jal  ra, rand_led  # 2) a5 = LED objetivo (1, 2, 4 u 8)
0000006c  00f42223      sw   a5, LED_OFF(s0)  #    solo el LED objetivo encendido
00000070  00c42703      lw   a4, CYC_OFF(s0)  # 3) inicio de la medicion (instruccion siguiente)
00000074  00c42583      lw   a1, CYC_OFF(s0)  #    se lee el contador ANTES de los botones:
00000078  00842503      lw   a0, BTN_OFF(s0)  #    si a0 != 0, a1 es el instante de la pulsacion
0000007c  fe050ce3      beqz a0, wait_press
00000080  40e585b3      sub  a1, a1, a4  # a1 = ciclos transcurridos (sin signo)
00000084  00b181b3      add  gp, gp, a1  # mezcla el tiempo de reaccion en la semilla
00000088  08f51c63      bne  a0, a5, wrong_button  # boton distinto (o varios) = error
0000008c  0ec092b7      li   t0, MAX_CYCLES
00000090  ce028293
00000094  0055e463      bltu a1, t0, no_sat
00000098  00028593      mv   a1, t0  # satura en 9,9 s para no desbordar la suma
                  no_sat:
0000009c  00b20233      add  tp, tp, a1  # acumula para el promedio
000000a0  001312b7      li   t0, HALF_TENTH
000000a4  2d028293
000000a8  00558533      add  a0, a1, t0  # a0 = ciclos + medio paso (redondeo)
000000ac  002625b7      li   a1, TENTH
000000b0  5a058593
000000b4  158000ef      jal  ra, udiv  # a0 = decimas = (ciclos + TENTH/2) / TENTH
```

```
000000b8  170000ef      jal  ra, to_bcd  # a0 = dos digitos decimales
000000bc  00a42023      sw   a0, SEG_OFF(s0)  # 5) mostrar tiempo en decimas
000000c0  0dc000ef      jal  ra, wait_release
000000c4  00148493      addi s1, s1, 1  # 6) una respuesta correcta mas
000000c8  00a00293      li   t0, NUM_ROUNDS
000000cc  f854c2e3      blt  s1, t0, round_start
000000d0  00bec2b7      li   t0, NUM_ROUNDS * TENTH / 2
000000d4  c2028293
000000d8  00520533      add  a0, tp, t0
000000dc  017d85b7      li   a1, NUM_ROUNDS * TENTH
000000e0  84058593
000000e4  128000ef      jal  ra, udiv
000000e8  140000ef      jal  ra, to_bcd
000000ec  00a42023      sw   a0, SEG_OFF(s0)
000000f0  00500613      li   a2, 5
                  end_loop:
000000f4  00c42223      sw   a2, LED_OFF(s0)
000000f8  004c5537      li   a0, IDLE_STEP
000000fc  b4050513
00000100  074000ef      jal  ra, wait_or_press
00000104  00051663      bnez a0, end_pressed
00000108  00f64613      xori a2, a2, 15  # 0101 <-> 1010
0000010c  fe9ff06f      j    end_loop
                  end_pressed:
00000110  00c42283      lw   t0, CYC_OFF(s0)
00000114  0051c1b3      xor  gp, gp, t0
00000118  084000ef      jal  ra, wait_release
0000011c  ef1ff06f      j    new_game
                  wrong_button:
00000120  0ee00293      li   t0, ERR_CODE
00000124  00542023      sw   t0, SEG_OFF(s0)
00000128  074000ef      jal  ra, wait_release
0000012c  00300613      li   a2, ERR_BLINKS
                  err_loop:
00000130  00f00293      li   t0, 15
00000134  00542223      sw   t0, LED_OFF(s0)
00000138  00394537      li   a0, BLINK_TIME
0000013c  87050513
00000140  020000ef      jal  ra, delay
00000144  00042223      sw   zero, LED_OFF(s0)
00000148  00394537      li   a0, BLINK_TIME
0000014c  87050513
00000150  010000ef      jal  ra, delay
00000154  fff60613      addi a2, a2, -1
00000158  fc061ce3      bnez a2, err_loop
0000015c  ef5ff06f      j    round_start
                  delay:
00000160  00c42283      lw   t0, CYC_OFF(s0)  # t0 = inicio
                  delay_loop:
00000164  00c42303      lw   t1, CYC_OFF(s0)
00000168  40530333      sub  t1, t1, t0  # transcurrido = ahora - inicio
0000016c  fea36ce3      bltu t1, a0, delay_loop
00000170  00008067      ret
                  wait_or_press:
00000174  00c42283      lw   t0, CYC_OFF(s0)
                  wop_loop:
00000178  00842303      lw   t1, BTN_OFF(s0)
0000017c  00031c63      bnez t1, wop_hit
```

```
00000180  00c42383      lw   t2, CYC_OFF(s0)
00000184  405383b3      sub  t2, t2, t0
00000188  fea3e8e3      bltu t2, a0, wop_loop
0000018c  00000513      li   a0, 0
00000190  00008067      ret
                    wop_hit:
00000194  00030513      mv   a0, t1
00000198  00008067      ret
                    wait_release:
0000019c  0007a6b7      li   a3, DEBOUNCE
000001a0  12068693
                    wr_restart:
000001a4  00c42283      lw   t0, CYC_OFF(s0)  # reinicia la ventana de tiempo
                    wr_loop:
000001a8  00842303      lw   t1, BTN_OFF(s0)
000001ac  fe031ce3      bnez t1, wr_restart  # algun boton sigue presionado (o rebotando)
000001b0  00c42383      lw   t2, CYC_OFF(s0)
000001b4  405383b3      sub  t2, t2, t0
000001b8  fed3e8e3      bltu t2, a3, wr_loop
000001bc  00008067      ret
                    rand_led:
000001c0  003182b3      add  t0, gp, gp  # 2*gp
000001c4  005282b3      add  t0, t0, t0  # 4*gp
000001c8  003281b3      add  gp, t0, gp  # 5*gp
000001cc  000032b7      li   t0, LCG_INC
000001d0  03928293
000001d4  005181b3      add  gp, gp, t0
000001d8  00c42283      lw   t0, CYC_OFF(s0)
000001dc  005181b3      add  gp, gp, t0  # + entropia del contador de ciclos
000001e0  00100793      li   a5, 1
000001e4  00010337      lui  t1, 0x10  # t1 = 0x0001_0000 (bit 16)
000001e8  0061f2b3      and  t0, gp, t1
000001ec  00028463      beqz t0, rl_bit17
000001f0  00f787b3      add  a5, a5, a5  # bit16 = 1 -> x2
                    rl_bit17:
000001f4  00630333      add  t1, t1, t1  # t1 = 0x0002_0000 (bit 17)
000001f8  0061f2b3      and  t0, gp, t1
000001fc  00028663      beqz t0, rl_done
00000200  00f787b3      add  a5, a5, a5  # bit17 = 1 -> x4
00000204  00f787b3      add  a5, a5, a5
                    rl_done:
00000208  00008067      ret
                    udiv:
0000020c  00000293      li   t0, 0
                    udiv_loop:
00000210  00b56863      bltu a0, a1, udiv_done
00000214  40b50533      sub  a0, a0, a1
00000218  00128293      addi t0, t0, 1
0000021c  ff5ff06f      j    udiv_loop
                    udiv_done:
00000220  00028513      mv   a0, t0
00000224  00008067      ret
                    to_bcd:
00000228  06300293      li   t0, MAX_TENTHS
0000022c  00a2f463      bgeu t0, a0, bcd_ok
00000230  00028513      mv   a0, t0
                    bcd_ok:
00000234  00000313      li   t1, 0  # decenas
```

```
00000238  00a00293      li   t0, 10
                  bcd_loop:
0000023c  00556863      bltu a0, t0, bcd_pack
00000240  ff650513      addi a0, a0, -10
00000244  00130313      addi t1, t1, 1
00000248  ff5ff06f      j    bcd_loop
                  bcd_pack:
0000024c  00630333      add  t1, t1, t1  # decenas * 2
00000250  00630333      add  t1, t1, t1  # decenas * 4
00000254  00630333      add  t1, t1, t1  # decenas * 8
00000258  00630333      add  t1, t1, t1  # decenas * 16  (equivale a << 4)
0000025c  00a30533      add  a0, t1, a0
00000260  00008067      ret
```